



SDEV2171

Lesson 11: Debugging and Developer Tools

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain why debugging is a normal part of development
- Use console logging to inspect state and runtime values
- Interpret common warnings and runtime symptoms
- Use development tools such as the Dev Menu, LogBox, and DevTools
- Apply a small-step debugging process to identify and resolve issues

Lesson Roadmap

1. Reconnect lesson-10 dynamic UI work to debugging
2. Define a simple debugging loop
3. Introduce logs, LogBox, and DevTools
4. Guided debugging lab
5. Testing and troubleshooting
6. Checkpoint and lesson-12 handoff

Starting Point From Lesson 10

- use the same Expo Router project from lesson 10
- confirm the home screen still opens the detail screen
- confirm the detail screen still adds items, clears items, and shows the empty state correctly

If your project is not ready

- if your project is `configured` , keep using your own project
- if your project is `partial` or `blocked` , recover the working baseline first
- if recovery will take too long, switch to the instructor-guided reference path and record your next step

Why Debugging Matters

- real apps rarely work perfectly on the first try
- bugs are normal, not evidence that development failed
- the goal is not guessing faster
- the goal is using evidence to narrow the problem and test one fix at a time

The Debugging Loop

1. reproduce the bug
2. name the symptom clearly
3. inspect the most relevant values
4. form one small hypothesis
5. change one thing
6. retest

Rule

- do not change five things and hope
- inspect first, then adjust

Common Evidence Sources

- what the screen shows
- what the screen does not show
- logs in the console
- LogBox warnings or errors
- DevTools inspection panels

Plain-language reminder

- the bug is giving you clues
- your job is to read the clues in order

Small Logging Pattern

```
console.log('studySteps length', studySteps.length);  
console.log('first item', studySteps[0]);
```

What this does

- shows the current value at runtime
- helps confirm whether your state matches the visible symptom

Good Logging Habits

- log the variable you are actually testing
- add a short label so the output is readable
- remove or reduce noisy logs after the bug is resolved
- avoid logging everything at once

Dev Menu And DevTools

- open the Dev Menu to reach development features
- open React Native DevTools to inspect logs and runtime behaviour
- use LogBox when warnings or errors appear in the app

Practical habit

- if the screen behaves strangely, check the visible symptom first
- then check logs or DevTools before rewriting code

Required Tool Moment

- every student should use at least one tool clue today, not only guesswork
- acceptable tool clues:
 - one exact LogBox or terminal error line
 - one DevTools Console observation
 - one Dev Menu or DevTools action that helped confirm the bug path

Why this matters

- lesson 11 is not only about fixing a bug
- it is also about choosing the right evidence source

How To Open The Main Tools

Developer menu

- if Expo is already running with a connected emulator, simulator, or USB device, try pressing `m` in the terminal
- Android emulator: `Cmd + M` on macOS or `Ctrl + M` on Windows or Linux
- iOS simulator: `Ctrl + Cmd + Z`

React Native DevTools

- open it from the Dev Menu, or
- press `j` in the terminal where Expo is running

LogBox

- watch the running app screen for warnings and errors
- read the exact message before editing code

Expo Go Versus Development Builds

- the course now shows both Expo Go and development builds earlier, but uses development builds as the standard path
- in current Android Bridgeless setups, Expo Go can leave React Native DevTools unreliable or unavailable
- lesson 11 should use the project's development-build path when DevTools is part of the debugging loop
- terminal logs and LogBox still matter, but DevTools assumptions should match the development-build runtime

Debugging Lab Shape

Working baseline

- a lesson-10 style dynamic-list screen

Instructor bug pattern

- one small bug at a time
- one clear symptom at a time
- one fix at a time

Student target

- explain the symptom
- inspect the right value
- apply one small fix

Known Bug Categories For Today

- list data shape bug
- rendering bug
- empty-state condition bug
- import or missing-tooling bug

Important boundary

- no advanced architecture debugging today
- stay close to lesson-10 list behaviour

Guided Debug Order

1. confirm the working lesson-10 baseline
2. reproduce one bug exactly
3. describe the symptom in one sentence
4. log or inspect the most relevant value
5. compare the symptom to the code
6. change one small thing
7. retest before touching anything else

Example Bug Reasoning

Symptom

- the list screen shows blank rows

First checks

- is the array shape correct?
- does `renderItem` read the right property?

Good next action

- log one item object
- compare its field names to the JSX in `renderItem`

One Full Example Loop

Reproduce

- add one new item after the instructor injects the wrong object shape bug

Inspect

- log the last item in `studySteps`

Compare

- check whether the new object uses `label` or a different property name

Fix

- restore the property name the screen actually renders

Retest

- add one more item and confirm the new row text appears

When To Compare With The Reference App

- use your own project first if it still runs
- switch to the instructor reference app only when the baseline is too broken
- compare one file or one section at a time

Compare order

- imports
- state
- handler
- render output

Common Debugging Mistakes

- editing multiple files before reproducing the bug
- adding logs without knowing what question they answer
- ignoring the exact error or warning text
- guessing at fixes without checking the relevant value first
- keeping noisy logs after the main issue is understood

Mid-Lab Checkpoint

Students should be ready to show

- one clearly named bug symptom
- one logged or inspected value
- one exact tool clue from LogBox, the terminal, or DevTools
- one small code change tied to a hypothesis

Students should also be able to answer

- What did you inspect first?
- Why was that the most relevant check?
- Why was that tool or clue better than guessing?
- What happened after the retest?

What We Learned Today

- debugging is a normal development activity
- symptoms, logs, and warnings are evidence
- console logs and DevTools help inspect runtime state
- one bug should lead to one small hypothesis and one retest
- this prepares us for debugging API and async behaviour in lesson 12

Lesson 12 Bridge

- next lesson adds remote data and async timing
- the same debugging loop will still matter
- new symptom types will appear:
 - a request runs more than once
 - data is not ready yet when the screen renders
 - a loading or error state appears at the wrong time

Keep from today

- reproduce
- inspect the most relevant clue
- change one thing
- retest

Next Step

- keep the lesson-10 list screen in a known-good state after the lab
- remove or reduce temporary logs that are no longer useful
- be ready to inspect runtime behaviour again when remote data enters the app next lesson